PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Transformer Based End-To-End Web Application Firewall (WAF) Pipeline

A PROJECT REPORT

Submitted by

KUSHAL V- 20221CBD0010

PRAVEEN R- 20221CBD0037

MAHESH BABU P- 20221CBD0054

Under the guidance of,

Dr. MANJULA HM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND TECHNOLOGY

(Big Data)

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "Transformer Based End-To-End Web Application Firewall (WAF) Pipeline" is a bonafide work of "Kushal V (20221CBD0010), Praveen R (20221CBD0037), Mahesh Babu P (20221CBD0054)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND TECHNOLOGY, BIG DATA during 2026-27.

21/4/26
Dr. Manjula HM
Associate Professor
Project Guide
PSCS (Big Data)
Presidency University

21/4/26
Dr. Manjula HM
Associate Professor
Program Project Coordinator
PSCS (Big Data)
Presidency University

Dr. Sampath A K
Professor
School Project Coordinator
PSCS (Big Data)
Presidency University

21/4/26
Dr. S Pravinth Raja
Professor & Head of the Department
PSCS (Big Data)
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS (Big Data)
Presidency University

Sl. No	Name of the Examiner	Signature	Date
	Dr Madheswaran M V		21/4/26
	Ms. Sandhya. L		21/04/26.

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (Big Data) at Presidency University, Bengaluru, named Kushal V, Praveen R , Mahesh Babu P , hereby declare that the project work titled **"Transformer Based End-To-End Web Application Firewall (WAF) Pipeline"** has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND TECHONOLOGY(Big Data) during the academic year of 2026-27. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Kushal V	20221CBD0010	
Praveen R	20221CBD0037	
Mahesh Babu P	20221CBD0054	

PLACE: BENGALURU

DATE: 26/04/2026

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr. Manjula HM, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. S Pravinth Raja , Professor & Head of the Department,** Presidency School of Computer Science and Engineering, Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering, Presidency University and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, PSCS** School Project Coordinators**, Dr. Manjula H M,** Program Project Coordinator, Presidency School of Computer Science and Engineering, Presidency University for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Kushal V

Praveen R

Mahesh Babu P

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Transformer Based End-To-End Web Application Firewall (WAF) Pipeline

A PROJECT REPORT

Submitted by

KUSHAL V- 20221CBD0010

PRAVEEN R- 20221CBD0037

MAHESH BABU P- 20221CBD0054

Under the guidance of,

Dr. MANJULA HM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND TECHNOLOGY

(Big Data)

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "Transformer Based End-To-End Web Application Firewall (WAF) Pipeline" is a bonafide work of "Kushal V (20221CBD0010), Praveen R (20221CBD0037), Mahesh Babu P (20221CBD0054)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND TECHNOLOGY, BIG DATA during 2026-27.

Dr. Manjula HM
Associate Professor
Project Guide
PSCS (Big Data)
Presidency University

Dr. Manjula HM
Associate Professor
Program Project Coordinator
PSCS (Big Data)
Presidency University

Dr. Sampath A K
Professor
School Project Coordinator
PSCS (Big Data)
Presidency University

Dr. S Pravinth Raja
Professor & Head of the Department
PSCS (Big Data)
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS (Big Data)
Presidency University

Sl. No	Name of the Examiner	Signature	Date

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (Big Data) at Presidency University, Bengaluru, named Kushal V, Praveen R , Mahesh Babu P , hereby declare that the project work titled **"Transformer Based End-To-End Web Application Firewall (WAF) Pipeline"** has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND TECHONOLOGY(Big Data) during the academic year of 2026-27. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Kushal V 20221CBD0010

Praveen R 20221CBD0037

Mahesh Babu P 20221CBD0054

PLACE: BENGALURU

DATE: 26/04/2026

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Dr. Manjula HM, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. S Pravinth Raja , Professor & Head of the Department,** Presidency School of Computer Science and Engineering, Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering, Presidency University and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, PSCS** School Project Coordinators**, Dr. Manjula H M,** Program Project Coordinator, Presidency School of Computer Science and Engineering, Presidency University for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Kushal V

Praveen R

Mahesh Babu P

ABSTRACT

Web applications are increasingly becoming susceptible to various complex cyber attacks such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Path Traversal, and Command Injection. Conventional signature-based Web Application Firewalls (WAFS) are largely rule and pattern-based making them ineffective against zero-day and obfuscated attacks. In response to these issues, the proposed research implements a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF), which uses deep learning-based methodologies to conduct real-time analysis of the HTTP request and classify threats based on sophisticated measures. The proposed system consists of a fine-tuned DistilBERT transformer model that is used to examine contextually enriched HTTP requests. Semantic features, special character patterns, and payload structural patterns are obtained with the aid of a custom Request Encoder before tokenization, which is needed to aid in contextual understanding. The transformed encoded input is then provided into the transformer model which contains self-attention mechanisms to identify complex relationships between the request elements. The model provides predictions regarding the probabilities of all the five types: SAFE, SQLI, XSS, PATHTRAVERSAL, and COMMANDINJECTION.A confidence-sensitive decision engine is provided to help with more trustworthy security decisions. The suggested system categorizes the threats as being of high, medium or low confidence in their regions using softmax-based confidence scores as opposed to conventional models that merely use predicted class labels. Very likely malicious requests are blocked immediately, somewhat likely malicious requests are recorded with warnings, and not very likely malicious ones are set to be analyzed manually. This multi layered decision making approach has the advantage of eliminating false positives but offers good threat coverage. The results of testing the proposed AI-WAF on curated and unseen sets of HTTP requests indicate that the given AI-WAF has the overall accuracy of 90.62% on unseen test data and a macro F1-score of 0.9125. In nearly all kinds of attacks, the system maintains an accuracy of 100 percent and in the critical injection-based attacks, the system reduces the number of false negatives significantly. Findings indicate that transformer-based contextual modeling using confidence-sensitive decision logic can be a robust, general, and scalable method of securing web applications in the present era.

Table of Content

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	**Artificial Intelligence**
ML	**Machine Learning**
DL	**Deep Learning**
WAF	**Web Application Firewall**
AI-WAF	**Artificial Intelligence Web Application Firewall**
HTTP	**HyperText Transfer Protocol**
SQL	**Structured Query Language**
SQLI	**SQL Injection**
XSS	**Cross-Site Scripting**
NLP	**Natural Language Processing**
BERT	**Bidirectional Encoder Representations from Transformers**
DistilBERT	**Distilled version of BERT**
API	**Application Programming Interface**
JSON	**JavaScript Object Notation**
CSV	**Comma-Separated Values**
REST	**Representational State Transfer**
UI	**User Interface**

URL	**Uniform Resource Locator**
IP	**Internet Protocol**
IDS	**Intrusion Detection System**
CNN	**Convolutional Neural Network**
RNN	**Recurrent Neural Network**
LSTM	**Long Short-Term Memory**
TP	**True Positive**
TN	**True Negative**
FP	**False Positive**
FN	**False Negative**
F1-score	**Harmonic Mean of Precision and Recall**
GPU	**Graphics Processing Unit**
CPU	**Central Processing Unit**
DoS	**Denial of Service**
OWASP	**Open Web Application Security Project**

CHAPTER 1

INTRODUCTION

With the rapid growth of web-based applications and digital services, web security has become a critical concern as applications are increasingly exposed to cyber threats such as SQL Injection, Cross-Site Scripting (XSS), Path Traversal, and Command Injection attacks. Traditional Web Application Firewalls (WAFs), which rely on predefined rules and signatures, are effective against known threats but struggle to detect obfuscated, modified, and zero-day attacks due to their lack of contextual understanding. To address these limitations, this project proposes a Transformer-Based AI Web Application Firewall that leverages a fine-tuned DistilBERT model to analyze HTTP requests as contextual text and classify them into multiple attack categories. The system incorporates a confidence-aware decision engine that determines whether to allow, block, or log requests based on prediction confidence, thereby improving reliability and reducing false positives. Implemented as a real-time middleware using FastAPI, the proposed solution integrates request parsing, encoding, model inference, decision-making, logging, and monitoring into a unified pipeline, demonstrating the effectiveness of transformer-based deep learning in enhancing modern web application security.

1.1 Background

The rapid growth of web applications and online services has significantly increased the importance of web application security. Web applications are widely used in banking, e-commerce, education, healthcare, and cloud services, making them a primary target for cyber attackers. Attackers often exploit vulnerabilities in web applications by injecting malicious payloads through HTTP requests, leading to data breaches, unauthorized access, and system compromise. Common web application attacks include SQL Injection, Cross-Site Scripting (XSS), Path Traversal, and Command Injection attacks.Traditional Web Application Firewalls (WAFs) rely on rule-based and signature-based detection techniques to identify malicious requests. However, these systems are not effective against new and obfuscated attack patterns because attackers frequently modify payload structures to bypass security rules. To address these limitations, Artificial Intelligence and Machine Learning techniques are increasingly being used in cybersecurity applications to detect malicious patterns automatically.

This project proposes a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF) that uses a DistilBERT transformer model to analyze HTTP requests and classify them into different attack categories. The system also includes a confidence-aware decision engine, logging system, and monitoring dashboard to provide real-time web application security. The objective of this project is to develop an intelligent web application firewall capable of detecting modern web attacks using deep learning techniques.

1.2 Scope and Limitations

The scope of this project is to design and implement an Artificial Intelligence-based Web Application Firewall capable of detecting common web application attacks using a transformer-based deep learning model. The system analyzes HTTP requests, extracts contextual features, classifies the requests into predefined attack categories, and makes security decisions based on prediction confidence.

The system focuses on detecting the following types of web attacks:

- SQL Injection
- Cross-Site Scripting (XSS)
- Path Traversal
- Command Injection
- Benign (SAFE) requests

The project includes dataset preparation, model training, model evaluation, and real-time deployment of the AI-WAF system using a middleware architecture and a monitoring dashboard interface. However, the system also has certain limitations. The model performance depends on the quality and diversity of the training dataset. The system currently supports only predefined attack categories and may require additional training to detect new attack types. The system is designed as a prototype and demonstration system and may require optimization for large-scale production deployment. Additionally, transformer models require computational resources, which may affect performance in high-traffic environments.

1.3 Problem Statement

Web applications are increasingly vulnerable to various cyber attacks such as SQL Injection, Cross-Site Scripting, Path Traversal, and Command Injection attacks. Traditional Web Application Firewalls rely on static rules and signature-based detection methods, which are ineffective against new, obfuscated, or zero-day attacks. Attackers can bypass rule-based systems by modifying attack

payloads or using encoding techniques.

Therefore, there is a need for an intelligent web application firewall that can analyze HTTP requests contextually, detect malicious patterns automatically, and make security decisions in real time. The problem addressed in this project is the development of a machine learning-based web application firewall that can classify HTTP requests into different attack categories and provide real-time protect

1.4 Motivation

The motivation for this project arises from the increasing number of cyber attacks targeting web applications and the limitations of traditional rule-based web application firewalls. Modern cyber attacks are becoming more sophisticated, and attackers frequently modify attack payloads to bypass security systems. Rule-based systems require continuous updates and manual rule creation, which is time-consuming and inefficient. Artificial Intelligence and Machine Learning techniques provide an opportunity to develop intelligent security systems that can learn attack patterns automatically from data. Transformer models such as DistilBERT have shown strong performance in understanding contextual relationships in text data. Since HTTP requests can be represented as textual data, transformer models can be used to analyze request patterns and detect malicious behavior The motivation of this project is to develop an intelligent AI-based Web Application Firewall that can automatically detect web attacks using deep learning techniques and improve web application security through intelligent request analysis.

1.5 Methodology Overview

methodology followed in this project includes dataset preparation, feature extraction, model training, evaluation, and real-time deployment of the AI-WAF system. First, a dataset of HTTP requests containing both benign and malicious samples is collected and preprocessed. The dataset is then split into training, validation, and testing sets. Next, a Request Encoder is developed to extract contextual information from HTTP requests, including request method, path, query parameters, payload structure, and suspicious patterns. The encoded request data is then tokenized and passed to a DistilBERT transformer model for classification. The transformer model analyzes the contextual representation of HTTP requests and predicts the probability of each attack category. A confidence-aware decision engine evaluates the prediction probabilities and determines whether the request should be allowed, blocked, or logged for monitoring. The system is deployed as a middleware layer

that intercepts HTTP requests in real time and displays logs and predictions through a monitoring dashboard interface.

1.6 Project Overview Summary

This project presents the design and implementation of a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF) for detecting and preventing web application attacks. The system uses a DistilBERT transformer model to classify HTTP requests into different attack categories based on contextual analysis. A confidence-aware decision engine is used to improve decision reliability by evaluating prediction confidence before blocking requests.

The AI-WAF system includes multiple components such as HTTP request parsing, request encoding, transformer-based classification, decision engine, logging system, and monitoring dashboard. The system is capable of analyzing HTTP requests in real time and detecting common web application attacks such as SQL Injection, Cross-Site Scripting, Path Traversal, and Command Injection attacks.

The project demonstrates that transformer-based deep learning models can be effectively used for web application security and intrusion detection. The proposed system provides an intelligent and adaptive approach to web security compared to traditional rule-based firewall systems.

CHAPTER 2

LITERATURE REVIEW

The rapid evolution of web technologies has led to an increase in sophisticated cyber threats, making web application security a critical area of research. Traditional Web Application Firewalls (WAFs), which rely on rule-based and signature-based detection mechanisms, have shown limitations in identifying modern attacks such as obfuscated payloads and zero-day exploits. As a result, recent research has focused on integrating machine learning and deep learning techniques for intelligent threat detection. In particular, transformer-based models have gained significant attention due to their ability to capture contextual relationships in textual data, making them highly effective for analyzing HTTP requests and detecting complex attack patterns. Several studies have explored the application of transformers in vulnerability detection, intrusion detection, and web security, demonstrating improved accuracy and adaptability compared to conventional approaches. This literature review examines existing methodologies, highlights their strengths and limitations, and provides insights into how the proposed transformer-based AI-WAF system advances the current state of web application security.

2.1 Introduction to Related Work

In recent years, web application security has become a major research area due to the rapid increase in cyber attacks targeting web applications and cloud services. Traditional Web Application Firewalls (WAFs) rely on rule-based and signature-based detection techniques, which are not effective against modern attacks such as obfuscated payloads, zero-day exploits, and polymorphic attacks. To overcome these limitations, researchers have explored machine learning and deep learning approaches for detecting malicious web traffic and software vulnerabilities.

Recent research has particularly focused on transformer-based models such as BERT, DistilBERT, RoBERTa, and GPT models for cybersecurity applications. These models are capable of understanding contextual relationships in data and have shown significant improvements in tasks such as vulnerability detection, malware detection, malicious URL classification, and intrusion detection systems. Transformer models can process raw security data such as HTTP requests, logs, and network traffic without relying heavily on handcrafted feature extraction methods.

Therefore, transformer-based approaches are increasingly being used to build intelligent and adaptive web security systems capable of detecting complex attack patterns.

2.2 Survey of Existing Papers

Thapa et al. [1] examined the use of transformer-based language model in detecting software vulnerability in C/C++ source code. Their model combines preprocessing, tokenization, embedding, and fine-tuning on models BERT, DistilBERT, RoBERTa, and CodeBERT. It has been shown that the experimental results are more precise and have better recall and F1-scores, which is why the attention mechanisms prove to be more effective when identifying complex vulnerability patterns.

Mamede et al. [2] suggested an IDE transformer-based VDet (Vulnerability detecting) plug-in to detect vulnerabilities in Java programs in real time. With JavaBERT refined to multi-label classification of CWE types, the system had reached 98.9% accuracy, which shows that it is indeed possible to directly apply transformer models to software development settings.

Rudd et al. [3] examined the possibility of using Transformers to the end-to-end tasks of InfoSec malicious URLs detection and malware categorization with raw Data in the form of a sequence of bytes. Their research demonstrated that the transformer architectures are capable of taking raw security data directly without feature engineering (although, the computational efficiency is still an issue).

Darwinkel [4] used transformer-encoded embeddings on HTTP response headers to perform fingerprinting of web servers, and high macro F1-scores were obtained in classification. This paper exhibits how transformers can extract the contextual patterns in the communication based on the HTTP protocol that is applicable to web traffic surveillance system.

Alshomrani et al. [5] have given an extensive review on transformer-based malware detection system, citing its superiority to the conventional signature-based systems in dealing with zero day and polymorphic attacks. On the same note, Latibari et al. [6] investigated security issues in transformer architectures where they addressed the issue of adversarial threats and the necessity to deploy strategies that are resistant to vulnerable devices.

The interpretable federated transformer framework proposed by De La Torre Parra et al. [7] focuses on explainable and privacy preserving AI in security systems, offering cloud log anomaly detection. Ravi et al. [8] suggested a classification framework of encrypted SaaS traffic using hybrid transformers in CASB architectures, which is flexible to unknown

services.

Rahman et al. [9] introduced an AI-enhanced conceptual framework of WAF that is meant to safeguard the critical infrastructure against zero-day exploits. Their CI-AWARE framework combines the transformer-based contextual examination with the layer of anomaly recognition to create a reference framework of AI-driven WAF frameworks.

To evaluate the attention-based models, Black et al. [10] put forward the FADE dataset, a large-scale benchmark of realistic HTTP payloads used to test firewall attacks, and enable evaluation of the models. Szabo and Bilicki [11] examined the topic of GPT based source code vulnerability inspection by showing the increased generalizability of large language models to security analysis.

There is a review of high-performance computing strategies in scalable transformer training by Hasan and Islam [12], which focuses on distributed optimization and cyber-resilient infrastructure. Ali [13] presented a threat detection system that is an AI-based system of detecting DNS over HTTPS when using advanced transformer-based architecture. Lastly, Ferla [14] has shown how machine learning models can be integrated into cloud WAF systems to detect bots, as well as classify HTTP traffic.

Taken together, these papers define the increasing role of transformer architectures in cybersecurity applications. Nevertheless, little studies have been made on the application of lightweight transformer-based WAF systems with real-time confidence-based decision mechanisms and visualization dashboards. This gap is filled by the proposed work that develops a production-focused Transformer-based AI WAF with structured logging, confidence-based blocking, and frontend interactive monitoring.

Table 2.1 Literature Review Summary

Article Title	**Published Year**	**Journal / Conference Name**	**Methods Used**	**Key Features**

Transformer-based language models for software vulnerability detection	2022	IEEE/ACM International Conference on Software Engineering	BERT, DistilBERT, RoBERTa, CodeBERT	Vulnerability detection using transformer models, dataset cleaning, multi-class classification
A transformer-based IDE plugin for vulnerability detection	2022	IEEE International Conference on Software Maintenance and Evolution	JavaBERT Transformer	IDE-based vulnerability detection, multi-label classification, real-time code analysis
Transformers for end-to-end InfoSec tasks: A feasibility study	2022	IEEE Security and Privacy Workshops	Transformer models for malware and URL classification	End-to-end security task learning from raw data, byte-level transformer
Interpretable federated transformer log learning for cloud threat forensics	2022	IEEE Transactions on Information Forensics and Security	Federated Learning + Transformer	Log anomaly detection, privacy-preserving security analytics
Utilizing GPT language models for source code inspection	2023	IEEE Access	GPT Models	Source code vulnerability detection using LLMs

Fingerprinting web servers through transformer-encoded HTTP response headers	2024	Computers & Security Journal	RoBERTa Transformer	HTTP header analysis, web server fingerprinting, deep learning classification
Survey of transformer-based malicious software detection systems	2024	IEEE Access	Transformer survey	Malware detection techniques using transformer architectures
Transformers: A security perspective	2024	IEEE Access	Security analysis of transformer models	Security risks, adversarial attacks, hardware and software vulnerabilities
Enhancing cloud-based WAF with machine learning models	2024	Master's Thesis, University of Padova	ML models for bot detection	Cloud-based WAF architecture, HTTP traffic classification
Hybrid transformer-based classification framework in enterprise cloud ecosystems	2025	IEEE Access	Random Forest + Transformer	Encrypted traffic classification, hybrid ML framework

Firewall attack detections and extractions (FADE)	2025	USENIX Security Symposium	Dataset + ML Models	Large HTTP attack dataset, benchmark for WAF research
AI-enhanced web application firewalls for protecting critical infrastructure	2026	IEEE Access	Deep Learning, Transformer -based WAF	AI-based WAF architecture, zero-day attack detection

2.3 Research Gaps Identified

Although many research works have been conducted in the field of cybersecurity using machine learning and transformer models, several research gaps still exist.

Most existing Web Application Firewalls are rule-based systems that rely on predefined attack signatures and patterns. These systems are unable to detect new or modified attack payloads. Machine learning based WAF systems have been proposed, but many of them rely on traditional machine learning algorithms such as Random Forest, SVM, and clustering methods, which cannot effectively capture contextual relationships in HTTP requests.

Some transformer-based security systems focus only on malware detection, vulnerability detection, or network intrusion detection, but very few studies focus specifically on transformer-based Web Application Firewalls for real-time HTTP request classification.

Another research gap is that many existing systems only perform classification but do not include a decision engine that considers confidence scores before blocking requests. This often leads to high false positive rates and unnecessary blocking of legitimate traffic.

Additionally, many research works do not provide real-time monitoring dashboards, logging systems, or middleware-based deployment architectures, which are important for practical implementation of a Web Application Firewall system.

Therefore, there is a need to develop a transformer-based AI Web Application Firewall that can perform real-time HTTP request classification, confidence-based decision making, logging, and monitoring in a practical deployment environment.

2.4 Objectives of the Study

The main objectives of this study are as follows:

The first objective is to design and develop an AI-based Web Application Firewall using a transformer-based deep learning model for detecting web application attacks.

The second objective is to develop a request encoding mechanism that converts HTTP requests into contextual text representations suitable for transformer models.

The third objective is to train and evaluate a DistilBERT transformer model for classifying HTTP requests into different attack categories such as SQL Injection, Cross-Site Scripting, Path Traversal, Command Injection, and Safe requests.

The fourth objective is to implement a confidence-aware decision engine that uses softmax probability scores to decide whether to allow, block, or log suspicious requests.

The fifth objective is to develop a logging and monitoring dashboard for real-time visualization of WAF activity and attack detection results.

The final objective is to integrate the trained model into a middleware-based Web Application Firewall system capable of performing real-time HTTP request inspection and attack detection.

2.5 Summary

This chapter presented a literature review of existing research related to transformer-based cybersecurity systems, vulnerability detection, malicious traffic detection, and AI-enhanced Web Application Firewalls. The literature survey showed that transformer models provide better contextual understanding and detection accuracy compared to traditional machine learning and rule-based security systems.

However, existing research works still lack integrated systems that combine transformer-based HTTP request classification, confidence-based decision making, real-time logging, and monitoring dashboards in a practical Web Application Firewall implementation. Based on these research gaps, this project proposes a Transformer-Based Artificial Intelligence Web Application Firewall that integrates a DistilBERT model, request encoding, decision engine, and monitoring dashboard for real-time web security.

CHAPTER 3

METHODOLOGY

The methodology of the proposed AI-based Web Application Firewall focuses on designing a systematic and end-to-end pipeline for detecting and preventing malicious HTTP requests using transformer-based deep learning. The approach begins with the collection and preprocessing of HTTP request data, followed by converting the requests into contextual text representations through a request encoding mechanism. These encoded inputs are then tokenized and used to train a DistilBERT model for multi-class classification of web attacks. The trained model is integrated into a real-time system where incoming requests are analyzed, classified, and assigned confidence scores. A confidence-aware decision engine is employed to determine whether to allow, block, or log requests based on prediction certainty. The system also incorporates logging and monitoring components to track performance and detected threats. This structured methodology ensures accurate detection, efficient real-time processing, and improved adaptability to modern web security challenges.

3.1 Research Methodology Outline

This research focuses on the design and development of a **Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF)** for detecting malicious HTTP requests. The proposed system uses machine learning and natural language processing techniques to classify web requests into attack categories and make security decisions in real time.

The overall research methodology follows a machine learning pipeline consisting of data collection, preprocessing, request encoding, tokenization, transformer-based classification, confidence-based decision making, logging, and dashboard monitoring. The system is designed to operate as a middleware firewall that intercepts HTTP requests before they reach the web application server.

The workflow begins with collecting HTTP request data containing both benign and malicious requests. The dataset is then preprocessed and encoded into contextual text format so that it can be processed by a transformer model. The encoded text is tokenized using a DistilBERT tokenizer and fed into the transformer model for classification. The model produces probability scores using a softmax function, and a decision engine uses these confidence scores to determine whether the request should be allowed, blocked, or logged. All requests and decisions are stored in logs and displayed on a real-time monitoring dashboard.

Fig 3.1 Overall Research Methodology

3.2 Detailed End-to-End Pipeline

The proposed AI-WAF system follows an end-to-end pipeline for detecting malicious HTTP requests and preventing web attacks. The system operates in multiple stages starting from HTTP request collection to final decision making and monitoring. First, HTTP request data is collected and stored in the dataset. The dataset includes both safe requests and malicious requests such as SQL Injection, Cross-Site Scripting, Path Traversal, and Command Injection attacks. The dataset is then preprocessed to clean the data and remove duplicate or invalid entries. After preprocessing, the dataset is split into training, validation, and test datasets. Next, the request encoder converts HTTP requests into contextual text representations. This step is important because transformer models work on text input, so HTTP requests must be converted into structured textual format. The encoded text is then tokenized using the DistilBERT tokenizer, which converts the text into token IDs and attention masks. These tokenized inputs are fed into the DistilBERT transformer model, which classifies the request into one of the attack categories. The model outputs probability scores for each class using the softmax function. The decision engine then uses these confidence scores to determine whether to allow or block the request. Finally, all requests and decisions are logged and displayed in the monitoring dashboard.

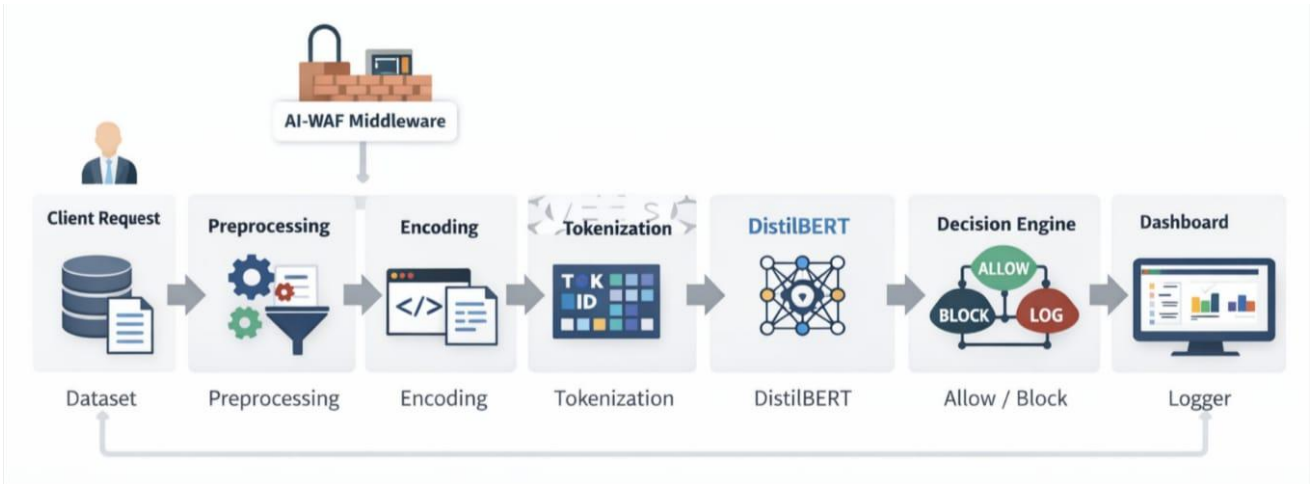


Fig 3.2 End-to-End Pipeline Diagram

3.3 Data Acquisition

The dataset used in this project consists of HTTP requests containing both benign and malicious payloads. The dataset includes different types of web attacks such as SQL Injection (SQLI), Cross-Site Scripting (XSS), Path Traversal, and Command Injection attacks.

The dataset was created by collecting HTTP request patterns and generating attack payloads manually and from web security resources. The dataset contains various HTTP request components such as HTTP method, URL path, query parameters, headers, and request body. The dataset is divided into training, validation, and testing datasets for model training and evaluation.

Table 3.1 Dataset Distribution Across Different Attack Classes

Class	Number of Samples
SAFE	25
SQLI	30
XSS	45
PATH_TRAVERSAL	20
COMMAND_INJECTION	27

3.4 Request Handling and Context Encoding

The AI-WAF system includes a request parser and request encoder module. The HTTP request parser extracts different components of the HTTP request such as HTTP method, URL path, query parameters, request body, and headers. The Request Encoder then converts these request components into a contextual text representation. The encoder combines structural information, special characters, payload patterns, and parameter values into a single textual representation. This contextual representation allows the transformer model to understand the relationships between different parts of the HTTP request and detect malicious patterns effectively. This step is very important because raw HTTP requests cannot be directly used by transformer models; they must be converted into contextual text format first. Additionally, the encoding process preserves the semantic meaning of the request while highlighting potential attack signatures embedded within it. It also ensures consistency in input format, which improves model learning and prediction accuracy. By transforming structured request data into natural language-like text, the model can leverage its contextual understanding capabilities more effectively. This enhances the system's ability to detect complex, obfuscated, and previously unseen attack patterns.

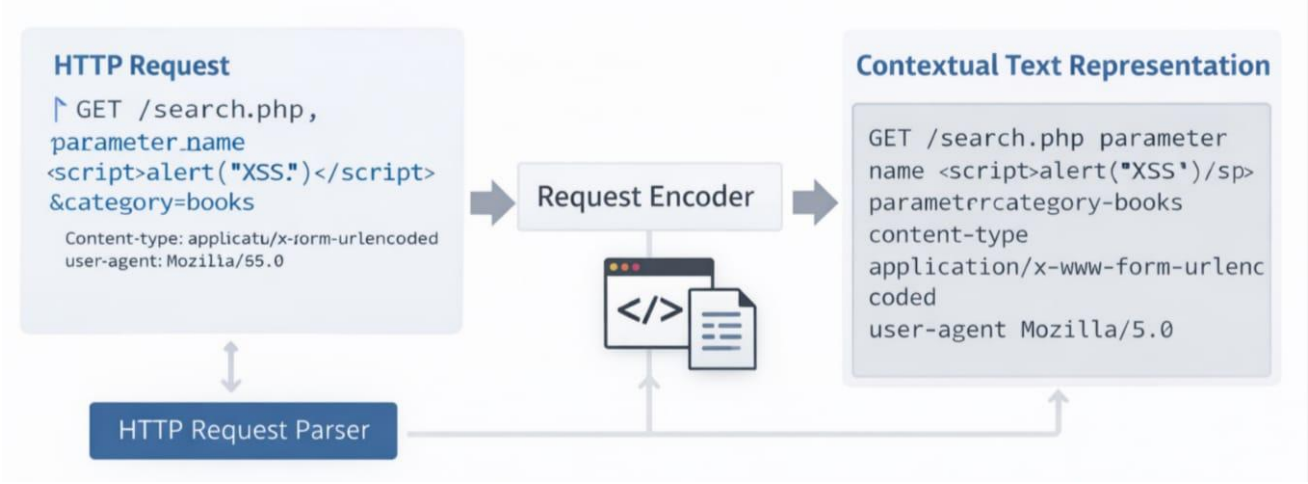


Fig 3.3 Request Encoding Process

3.5 Feature Extraction and Tokenization

After request encoding, the encoded text is tokenized using the DistilBERT tokenizer. Tokenization is the process of converting text into tokens that the transformer model can understand. The tokenizer converts text into token IDs and attention masks.

Token IDs represent the vocabulary index of each token, and the attention mask indicates which tokens should be attended to by the transformer model. These tokens are then converted into embeddings and passed to the transformer encoder layers. The self-attention mechanism in the transformer helps the model understand relationships between tokens and detect malicious patterns in HTTP requests.

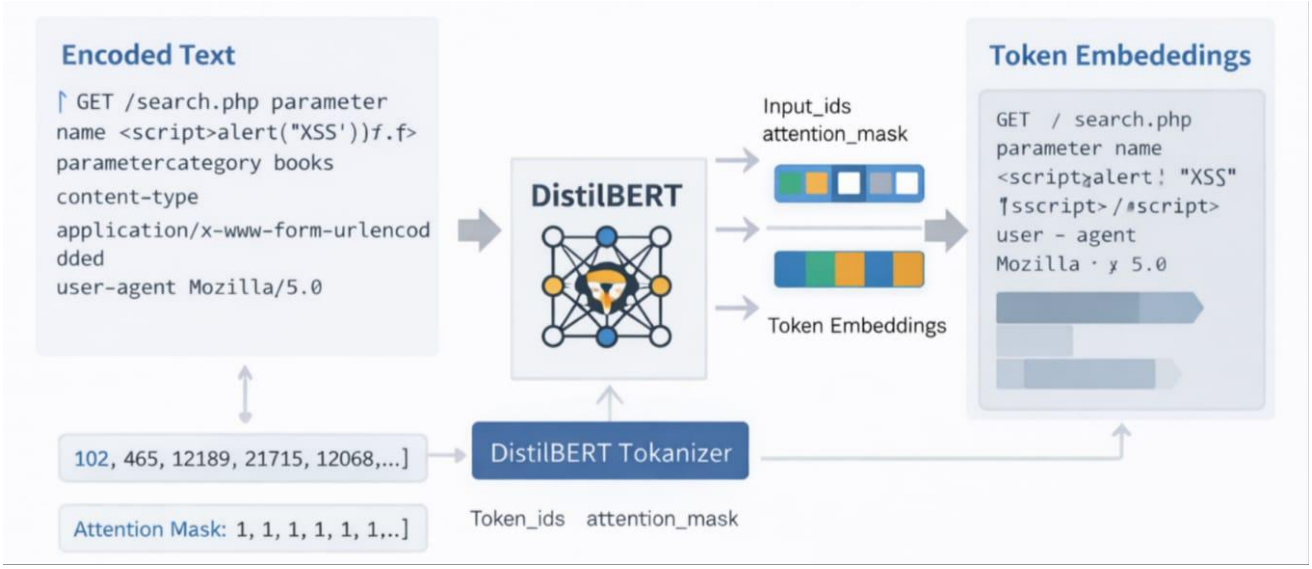


Fig 3.4 Request Encoding Process

3.6 Preprocessing Pipeline

The preprocessing pipeline includes multiple steps to prepare the dataset for model training. These steps include cleaning HTTP request data, encoding special characters, removing duplicate requests, and splitting the dataset into training, validation, and test datasets.

The dataset is then converted into encoded text format and tokenized inputs for the transformer model. Proper preprocessing is important to improve model accuracy and prevent overfitting.

Fig 3.5 Data Preprocessing Pipeline

3.7 Model Architecture

The proposed AI-WAF system uses a DistilBERT transformer model for HTTP request classification. The model consists of token embedding layers, transformer encoder layers with self-attention mechanisms, and a classification head. The classification head outputs class probabilities using the softmax function. DistilBERT is a lightweight version of BERT that provides faster training and inference while maintaining high accuracy, making it suitable for real-time web application firewall systems. The self-attention mechanism enables the model to focus on important parts of the input sequence, allowing it to capture relationships between different tokens in the request. This capability is particularly useful for identifying hidden or obfuscated attack patterns within HTTP requests. The reduced model size of DistilBERT ensures lower computational cost and faster response time, which is essential for real-time security applications. Furthermore, the model can be fine-tuned on domain-specific datasets to improve detection accuracy for various attack types. Overall, the use of DistilBERT provides an efficient balance between performance, speed, and scalability in the AI-WAF system.

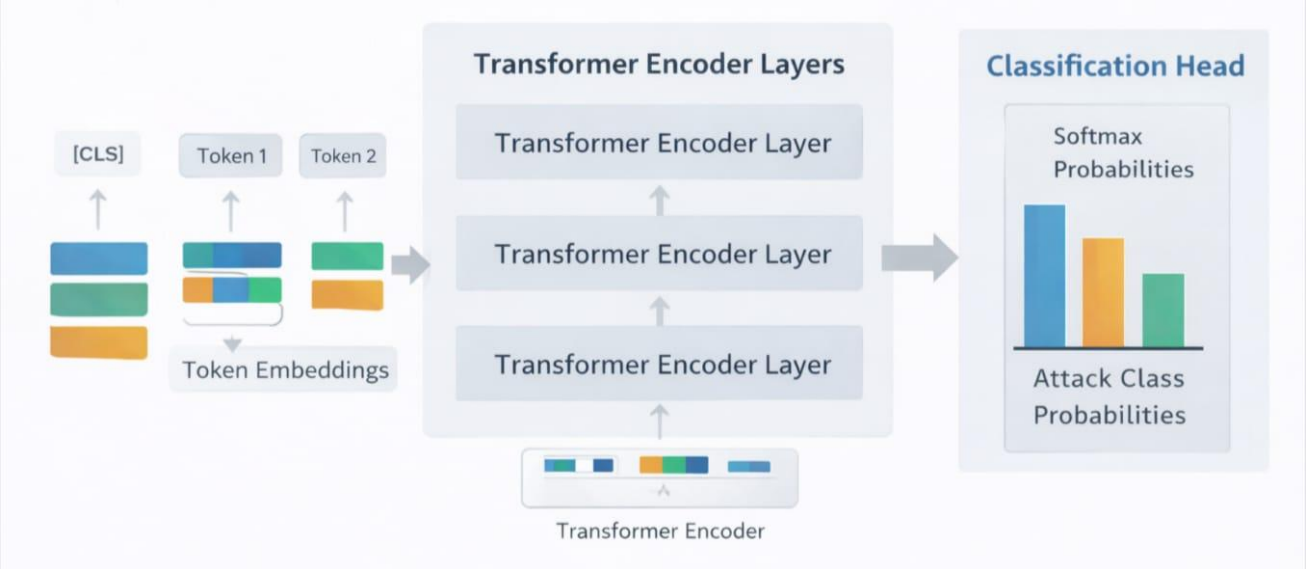


Fig 3.6 DistilBERT Model Architecture

Table 3.2 Model Training Parameters

Parameter	**Value**
Model	DistilBERT (distilbert-base-uncased)
Max Sequence Length	512
Batch Size	16
Learning Rate	2e-5
Epochs	3
Optimizer	AdamW

3.8 Training Strategy

The model is trained using cross-entropy loss and the AdamW optimizer. The training process includes forward pass, loss calculation, backpropagation, and weight updates. Early stopping and model checkpointing are used to prevent overfitting and to save the best model based on validation performance. The training process is repeated for multiple epochs until the model achieves optimal performance.

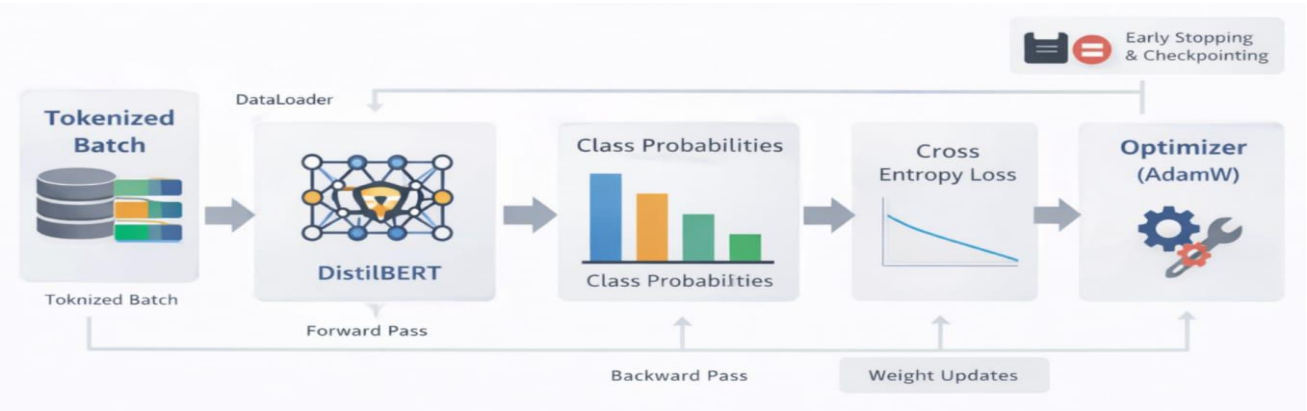


Fig 3.7 Model Training Process

3.9 Confidence-Aware Decision Engine

After classification, the softmax probability scores are used to determine the confidence level of the prediction. Based on the confidence score, the system decides whether to allow, block, or log the request.

For example:

- High-confidence attacks → Block request
- Medium-confidence attacks → Log request
- Low-confidence requests → Allow request

This confidence-based decision engine improves the reliability of the AI-WAF system and reduces false positives.

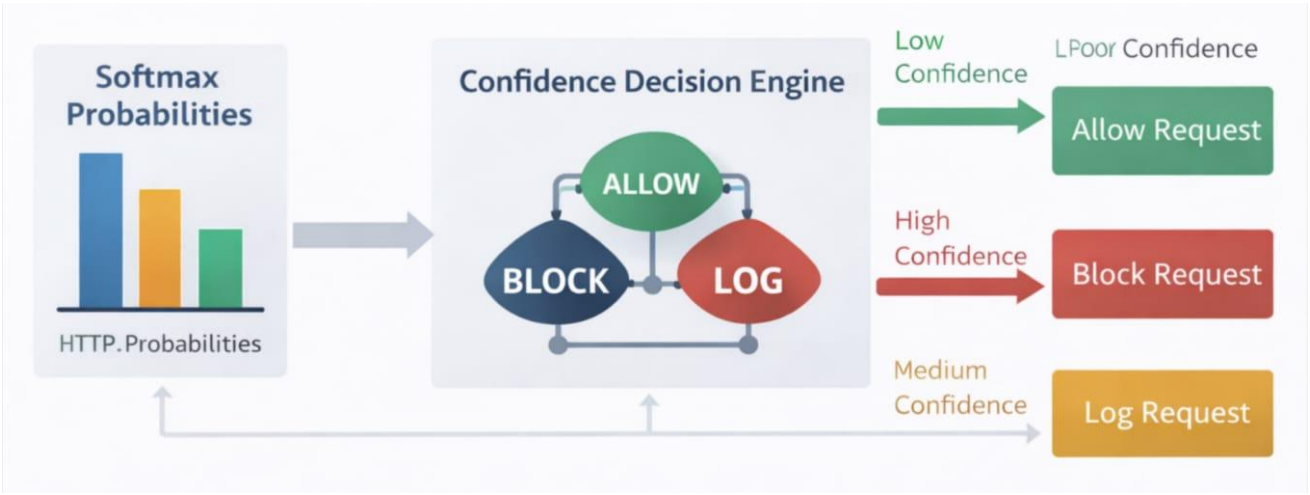


Fig 3.8 Confidence Decision Engine

3.10 Evaluation Metrics

The model performance is evaluated using accuracy, precision, recall, F1-score, and confusion matrix. These metrics help measure the performance of the classification model and the effectiveness of the AI-WAF system.

Table 3.3 Evaluation Metrics Used for Performance Analysis

Metric	Formula
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1 Score	PR / (P + R)

CHAPTER 4

IMPLEMENTATION

The implementation of the proposed AI-based Web Application Firewall focuses on integrating multiple components into a unified and functional system capable of real-time threat detection. The system is developed using a modular architecture that includes request parsing, contextual encoding, transformer-based classification, decision-making, logging, and monitoring. The DistilBERT model is implemented using PyTorch and HuggingFace Transformers for efficient training and inference, while FastAPI is used to deploy the system as a middleware that intercepts and analyzes incoming HTTP requests. Each component is designed to work seamlessly with others, ensuring efficient data flow from request processing to final decision output. The implementation also incorporates advanced techniques such as confidence-based decision thresholds, logging mechanisms, and performance monitoring to enhance reliability and usability. This structured implementation enables the system to operate effectively in real-world environments while maintaining scalability and low latency.

4.1 Module-Wise Implementation Details

The proposed Transformer-Based AI Web Application Firewall (AI-WAF) system is implemented as a modular architecture where each component performs a specific task in the request processing pipeline. This modular design improves scalability, maintainability, and ease of integration.

The first module is the **Request Parser**, which is responsible for extracting relevant information from incoming HTTP requests. It processes the HTTP method, URL path, query parameters, headers, and request body, and converts them into a structured format. This structured data is then passed to the Request Encoder, which generates a contextual textual representation of the request. The encoded representation captures both structural and semantic information, enabling the transformer model to understand relationships within the request.

The second module is the **Model Inference Module**, which uses a fine-tuned DistilBERT transformer model to classify the encoded request. The input text is tokenized using the DistilBERT tokenizer, which converts it into token IDs and attention masks. These inputs are passed through the transformer model to generate logits, which are then converted into probability scores using the

softmax function. The output includes the predicted attack class, confidence score, and probability distribution across all classes.

The third module is the **Decision Engine**, which plays a crucial role in translating model predictions into actionable security decisions. Instead of directly blocking requests based on predictions, the system uses a confidence-aware mechanism. High-confidence malicious predictions result in immediate blocking, medium-confidence predictions are logged for monitoring, and low-confidence predictions are allowed with a flag for manual review. This approach reduces false positives and improves system reliability.

The fourth module is the **Logging System**, which records all processed requests along with their predictions, confidence scores, decisions, threat levels, and latency. Logs are stored in a structured JSON format and are used for monitoring, analysis, and future model improvements. The logging module also supports real-time retrieval of logs for display in the dashboard.

The final module is the **FastAPI-based WAF Middleware**, which integrates all components into a real-time system. It intercepts incoming HTTP requests, processes them through the pipeline, and returns appropriate responses such as allowing or blocking requests. It also provides APIs for health monitoring, statistics, and log retrieval.

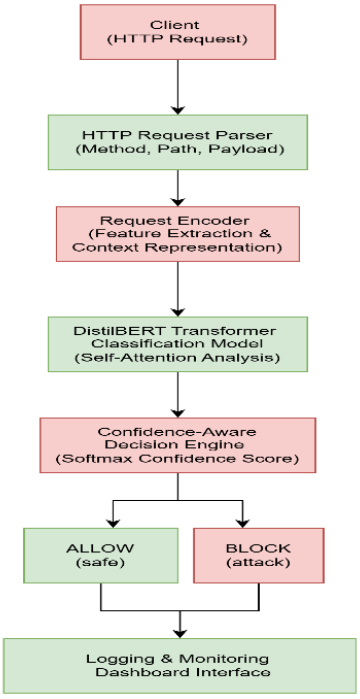


Fig 4.1 System Architecture

4.2 Algorithms

The implementation of the proposed AI-WAF system involves multiple algorithms that operate across different stages, including training, inference, decision-making, and evaluation. These algorithms work together to enable accurate detection and real-time prevention of malicious HTTP requests.

4.2.1 Training Algorithm

The training algorithm is responsible for fine-tuning the DistilBERT model using labeled HTTP request data. The process begins by loading preprocessed datasets consisting of encoded HTTP requests and corresponding labels. The input text is tokenized into token IDs and attention masks using the DistilBERT tokenizer. These tokenized inputs are then passed to the transformer model to generate predictions.

A weighted cross-entropy loss function is used to handle class imbalance by assigning higher weights to attack classes. The loss is minimized using the AdamW optimizer, and a linear learning rate scheduler with warmup is applied to improve convergence. Gradient clipping is used to stabilize training, and early stopping is implemented to prevent overfitting. The best-performing model is saved using checkpointing based on validation loss.

Algorithm: Model Training

Input: Training dataset D_train, Validation dataset D_val

Output: Trained model M

Begin

Initialize DistilBERT model

Initialize tokenizer

Set optimizer (AdamW) and scheduler

For each epoch do

For each batch in D_train do

Tokenize input text

Generate predictions using model

Compute weighted cross-entropy loss

Perform backpropagation

Update model parameters

Apply gradient clipping
End For

Evaluate model on D_val
If validation loss improves then
Save best model
Else
Increase patience counter
End If

If patience limit reached then
Stop training
End If
End For

Return trained model M
End

4.2.2 Inference Algorithm

The inference algorithm performs real-time classification of incoming HTTP requests. The request is first encoded into a textual format and then tokenized. The tokenized input is passed through the trained DistilBERT model to obtain logits, which are converted into probability scores using the softmax function. The class with the highest probability is selected as the predicted label, and the corresponding probability is considered the confidence score.

Algorithm: Request Classification

Input: Encoded HTTP request R
Output: Predicted label L, Confidence score C

Begin
Tokenize request R
Pass tokens to trained model
Compute logits
Apply softmax to get probabilities

L ← class with highest probability

C ← maximum probability

Return L, C

End

4.2.3 Confidence-Based Decision Algorithm

The decision-making algorithm determines whether a request should be allowed or blocked based on the predicted label and confidence score. If the request is classified as safe, it is immediately allowed. For attack predictions, the system compares the confidence score against predefined thresholds to decide the appropriate action.

Algorithm: Decision Engine

Input: Predicted label L, Confidence score C

Output: Action A

Begin

If L == SAFE then

A ← ALLOW

Else

If C ≥ 0.85 then

A ← BLOCK

Else if C ≥ 0.60 then

A ← ALLOW_WITH_LOG

Else

A ← ALLOW_WITH_FLAG

End If

End If

Return A

End

4.2.4 Logging Algorithm

The logging algorithm records all processed requests along with their predictions, decisions, and metadata. This helps in monitoring system performance and analyzing attack patterns.

Algorithm: Logging Process

Input: Request data R, Prediction P, Decision D, Latency T
Output: Log entry stored in file

Begin
  Create log entry with:
    timestamp
    request details
    predicted label
    confidence score
    decision taken
    threat level
    latency

  Append log entry to JSON log file
End

4.2.5 Evaluation Algorithm

The evaluation algorithm measures the performance of the trained model using test data. It computes various metrics such as accuracy, precision, recall, and F1-score. It also generates confusion matrices and performs error analysis.

Algorithm: Model Evaluation

Input: Test dataset D_test
Output: Evaluation metrics

Begin
  For each sample in D_test do
    Predict label using model
    Compare with true label
  End For

  Compute accuracy

Compute precision, recall, F1-score

Generate confusion matrix

Calculate false positives and false negatives

Return evaluation metrics

End

4.3 Pseudo Code

The overall working of the AI-WAF system can be described using the following pseudo code:

Algorithm: AI-WAF Request Processing

Input: HTTP Request

Output: Allow or Block Decision

Begin

Receive HTTP request

Parse request into structured components

Encode request into contextual text

Tokenize encoded text

Pass tokens to DistilBERT model

Compute logits and apply softmax

Get predicted label and confidence score

If label == SAFE then

Allow request

Else

If confidence ≥ high_threshold then

Block request

Else if confidence ≥ medium_threshold then

Allow request with logging

Else

```
        Allow request with flag for review
    End If
  End If

  Log request details, prediction, and decision
  Return response to client
End
```

4.4 Integration of Components

middleware acts as the central controller that connects all modules. When a client sends an HTTP request, the middleware intercepts it before it reaches the application server. The request is first passed to the Request Parser, which extracts and structures the request data. The structured data is then forwarded to the Request Encoder, which converts it into a textual format suitable for the transformer model.

The encoded text is processed by the Inference Module, which performs classification using the trained DistilBERT model. The output of the model, including the predicted label and confidence score, is passed to the Decision Engine. The Decision Engine evaluates the prediction and determines the appropriate action based on predefined thresholds. The final decision is then sent back to the middleware, which either blocks the request or allows it to proceed.

Simultaneously, the Logging Module records all relevant information, including request details, predictions, decisions, and latency. These logs are stored and made available to the frontend dashboard through API endpoints.

The dashboard provides real-time visualization of system activity, enabling users to monitor threats and system performance. This integration ensures that all components work together efficiently to provide a real-time, intelligent web application firewall.

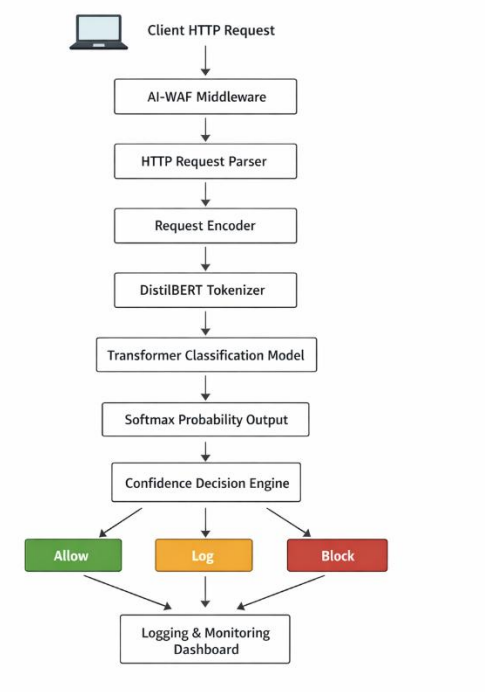


Fig. 4.2 Data flow diagram of AI-WAF request processing pipeline.

4.5 Summary

In this chapter, the implementation details of the Transformer-Based AI Web Application Firewall have been presented. The system is designed using a modular architecture consisting of request parsing, encoding, transformer-based classification, confidence-aware decision making, logging, and real-time deployment using FastAPI. Advanced techniques such as weighted loss, learning rate scheduling, early stopping, and confidence-based decision thresholds are incorporated to improve performance and reliability. The integration of these components results in a robust, scalable, and intelligent web security system capable of detecting and preventing modern web attacks.

Chapter 5

RESULTS AND DISCUSSION

This chapter presents the experimental results and analysis of the proposed Transformer-Based AI Web Application Firewall, evaluating its effectiveness in detecting and classifying malicious HTTP requests. The performance of the system is assessed using standard evaluation metrics such as accuracy, precision, recall, and F1-score, along with confusion matrix and error analysis. The results obtained from testing on both structured test data and unseen real-world-like requests demonstrate the model's ability to generalize and detect various attack types with high reliability. In addition to quantitative evaluation, the behavior of the confidence-based decision engine is analyzed to understand how prediction confidence influences security decisions such as allowing, blocking, or logging requests. The discussion further interprets the results, compares the proposed system with traditional and machine learning-based approaches, and highlights the strengths and limitations of the model in real-time web security applications.

5.1 Experimental Setup and Environment

The proposed AI-based Web Application Firewall system was implemented and evaluated using a deep learning framework built on PyTorch and HuggingFace Transformers. The DistilBERT model (distilbert-base-uncased) was fine-tuned on a dataset of HTTP requests containing both benign and malicious payloads. The dataset was divided into training, validation, and test sets, and additional unseen requests were included during evaluation to ensure realistic performance assessment.

The training process was conducted using a batch size of 16, a learning rate of 3×10^{-5}, and a total of 8 epochs. The AdamW optimizer was used along with a linear learning rate scheduler with warmup steps. Early stopping was applied with a patience of 5 epochs to prevent overfitting. Gradient clipping was also used to stabilize training.

The experiments were executed on a system with CPU/GPU support depending on availability, and the model dynamically selected the appropriate device. The inference system was deployed using FastAPI to simulate real-time web request handling. The system also recorded latency for each request to evaluate real-time performance.

Table 5.1 Experimental Setup Configuration

Parameter	Value
Model	DistilBERT (distilbert-base-uncased)
Task Type	Multi-class Classification
Number of Classes	5 (SAFE, SQLI, XSS, PATH_TRAVERSAL, COMMAND_INJECTION)
Dataset	Preprocessed HTTP Requests Dataset
Train/Validation/Test Split	Used (train.csv, val.csv, test.csv + unseen data)
Batch Size	16
Learning Rate	3×10^{-5}
Number of Epochs	8
Optimizer	AdamW
Learning Rate Scheduler	Linear Scheduler with Warmup
Warmup Steps	100
Weight Decay	0.01
Loss Function	Weighted Cross-Entropy Loss
Gradient Clipping	1.0
Early Stopping	Enabled (Patience = 5, Min Delta = 0.001)
Maximum Sequence Length	512
Tokenizer	DistilBERT Tokenizer (Fast)
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score
Additional Evaluation	Confusion Matrix, FP/FN Analysis, Confidence Distribution
Hardware	CPU / GPU (CUDA if available)
Frameworks Used	PyTorch, HuggingFace Transformers
Deployment Framework	FastAPI
Logging	JSON-based logging system
Inference Mode	Real-time + Batch inference supported

5.2 Performance Metrics Used

The performance of the proposed AI-WAF system was evaluated using standard classification metrics including accuracy, precision, recall, and F1-score. Accuracy measures the overall correctness of the model, while precision evaluates how many predicted attacks are actually correct. Recall measures the ability of the model to detect actual attacks, and F1-score provides a balanced measure of precision and recall.

In addition to these metrics, a confusion matrix was generated to analyze the distribution of correct and incorrect predictions across different attack classes. False positive and false negative rates were also calculated, as these are critical in security systems. False positives indicate benign requests incorrectly classified as attacks, while false negatives represent malicious requests incorrectly classified as safe, which pose a higher risk.

The system also analyzed confidence scores generated by the softmax function. Confidence distribution plots were used to understand how certain the model is in its predictions, which directly supports the confidence-based decision engine

Table 5.2 Evaluation Metrics Formula

Metric	Formula	Description
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Measures overall correctness of the model by calculating the ratio of correctly predicted samples to total samples.
Precision	TP / (TP + FP)	Measures how many predicted positive (attack) cases are actually correct.
Recall (Sensitivity)	TP / (TP + FN)	Measures how many actual positive (attack) cases are correctly identified by the model.
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	Harmonic mean of precision and recall, providing a balance between them.

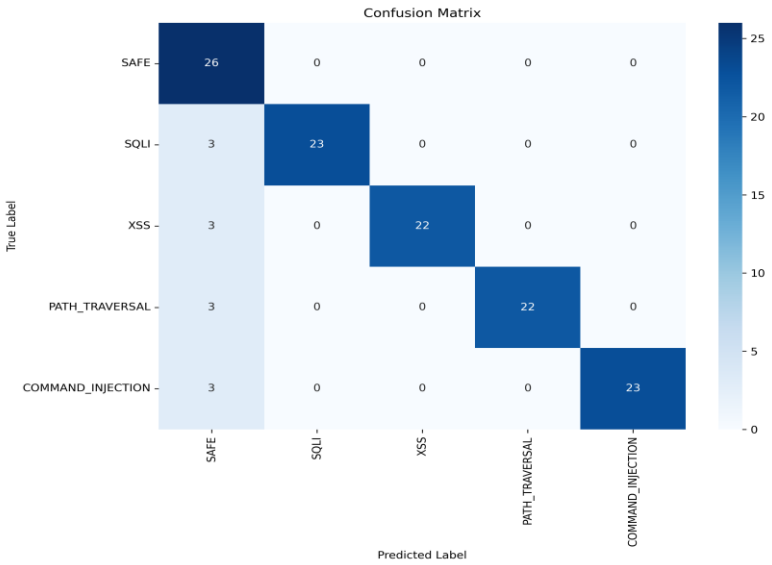


Fig 5.1 Confusion Matrix

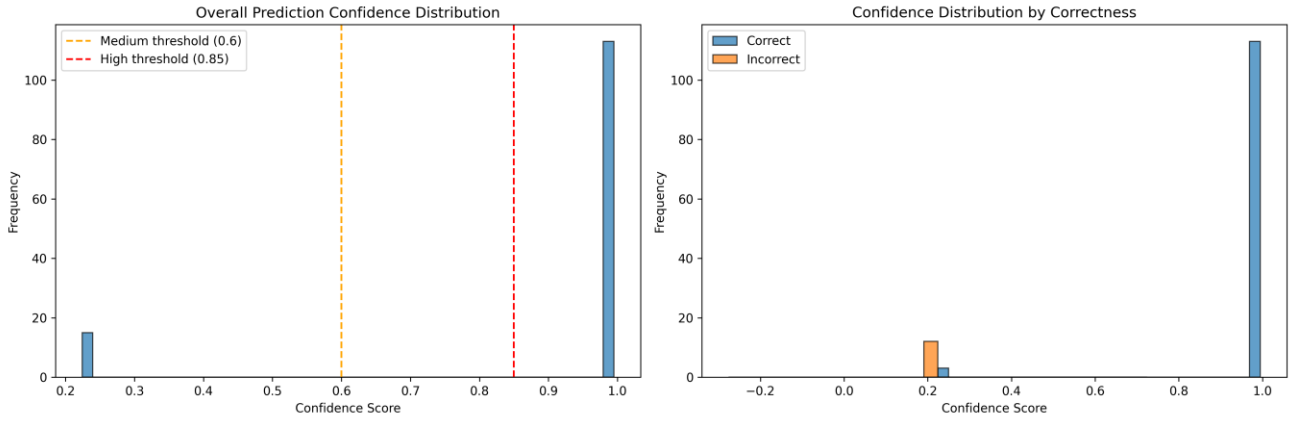


Fig 5.2 Confidence Distribution Graph

5.3 Comparative Results

The proposed transformer-based AI-WAF model achieved an overall accuracy of approximately 90–91% on unseen HTTP request data. This performance demonstrates a significant improvement over traditional rule-based WAF systems, which often fail to detect obfuscated or modified attack payloads.

Compared to conventional machine learning models such as Support Vector Machines (SVM) or Decision Trees, the DistilBERT model provides better contextual understanding of HTTP requests due to its self-attention mechanism. This allows it to capture relationships between different parts of the request, improving detection of complex attack patterns.

The use of unseen test data further validates the robustness of the model, as it ensures that the model is not overfitting to the training dataset. The results show that the model maintains consistent performance even when exposed to new and modified attack patterns.

Table 5.3 Comparative Results

Criteria	**Traditional WAF (Rule-Based)**	**Machine Learning Models (SVM, DT, etc.)**	**Proposed AI-WAF (DistilBERT)**
Detection Approach	Signature/Rule-Based	Feature-Based Learning	Contextual Deep Learning (Transformer)
Ability to Detect Zero-Day Attacks	Very Low	Moderate	High
Handling Obfuscated Attacks	Poor	Limited	Strong
Context Understanding	No	Partial	High (Self-Attention Mechanism)
Adaptability	Low (Manual Updates Required)	Moderate (Retraining Required)	High (Fine-tuning possible)
Accuracy	Low to Moderate (~60–75%)	Moderate (~75–85%)	High (~90–91%)
False Positive Rate	High	Moderate	Low
False Negative Rate	High	Moderate	Low
Real-Time Capability	High	Moderate	High (Optimized Inference)
Scalability	Limited	Moderate	High
Maintenance Effort	High (Rule Updates)	Moderate	Low (Model-based

			updates)
Decision Mechanism	Static Rules	Model Prediction Only	Confidence-Aware Decision Engine
Explainability	High (Rules visible)	Moderate	Moderate (Confidence + Scores)

5.4 Analysis and Interpretation of Results

The results indicate that the transformer-based approach is highly effective in detecting various types of web attacks, including SQL Injection, Cross-Site Scripting, Path Traversal, and Command Injection. The confusion matrix shows that most predictions are correctly classified, with only a small number of misclassifications occurring between similar attack categories.

The analysis of false positives and false negatives reveals that the system maintains a low false negative rate, which is critical for security applications. While some false positives are observed, these are mitigated by the confidence-based decision engine, which logs medium-confidence predictions instead of blocking them immediately.

The confidence distribution analysis shows that correct predictions generally have higher confidence scores, while incorrect predictions tend to have lower confidence. This validates the effectiveness of using confidence thresholds in the decision-making process. The thresholds of 0.85 and 0.60 provide a good balance between security and usability.

5.5 Discussion on Outcomes

The experimental results demonstrate that the proposed AI-WAF system successfully addresses the limitations of traditional rule-based firewalls. By leveraging transformer-based deep learning, the system is capable of understanding contextual relationships within HTTP requests and detecting complex attack patterns that are difficult to capture using static rules.

One of the key strengths of the system is the integration of a confidence-aware decision engine, which enhances reliability by avoiding unnecessary blocking of legitimate traffic. This approach ensures that high-confidence threats are blocked immediately, while uncertain cases are logged for further analysis.

The real-time implementation using FastAPI further demonstrates the practical applicability of the system. The measured inference latency indicates that the system can operate efficiently in real-world environments without introducing significant delays.

However, the system also has some limitations. The performance depends on the quality and diversity of the dataset, and new attack patterns may require retraining of the model. Additionally,

while DistilBERT provides good performance, more advanced transformer models could further improve accuracy at the cost of increased computational requirements.

5.6 Summary

In this chapter, the experimental results and performance analysis of the proposed AI-WAF system were presented. The system achieved high accuracy and demonstrated strong generalization capability on unseen data. The use of transformer-based deep learning, combined with a confidence-aware decision mechanism, provides a robust and intelligent approach to web application security. The results confirm that the proposed system is effective in detecting and mitigating modern web attacks while maintaining real-time performance.

CHAPTER 6

CONCLUSION AND FUTURE WORK

This chapter summarizes the overall outcomes of the proposed Transformer-Based AI Web Application Firewall and reflects on its effectiveness in addressing modern web security challenges. It consolidates the key contributions of the system, including the use of transformer-based deep learning for contextual request analysis and the implementation of a confidence-aware decision mechanism for reliable threat detection. The chapter also discusses the major findings obtained from experimental evaluation, highlighting the strengths of the model in achieving high accuracy and real-time performance. Furthermore, the limitations of the current approach are identified, providing a basis for future improvements. Finally, potential directions for future research are outlined to enhance the scalability, adaptability, and effectiveness of AI-driven web security systems.

6.1 Summary of Contributions

This research presented the design and implementation of a Transformer-Based Artificial Intelligence Web Application Firewall (AI-WAF) for detecting and preventing web-based attacks. The proposed system addresses the limitations of traditional rule-based firewalls by leveraging deep learning techniques for intelligent request classification. A complete end-to-end pipeline was developed, including dataset preparation, request encoding, transformer-based model training, evaluation, and real-time deployment.

One of the major contributions of this work is the use of the DistilBERT transformer model to classify HTTP requests into multiple attack categories, including SQL Injection, Cross-Site Scripting, Path Traversal, and Command Injection. The system introduces a contextual request encoding mechanism that enables the model to understand relationships within HTTP requests, improving detection accuracy.

Another significant contribution is the development of a confidence-aware decision engine, which enhances system reliability by making decisions based not only on predictions but also on confidence scores. The integration of logging and a real-time monitoring dashboard further strengthens the system by enabling continuous analysis and visualization of detected threats. Additionally, the system

was implemented as a FastAPI-based middleware, demonstrating its practical applicability in real-world environments.

6.2 Key Findings

The experimental results demonstrate that the proposed AI-WAF system achieves high accuracy and strong generalization capability on unseen HTTP request data. The use of transformer-based architecture enables the system to capture contextual relationships within requests, resulting in improved detection of complex and obfuscated attack patterns.

The analysis of evaluation metrics shows that the model maintains a balanced performance across precision, recall, and F1-score, indicating reliable classification across different attack types. The confusion matrix analysis reveals that most predictions are correctly classified, with only minor misclassifications between similar attack categories.

The confidence-based decision mechanism proves to be highly effective in reducing false positives while maintaining strong detection capability. High-confidence malicious requests are accurately blocked, while uncertain cases are logged for further analysis. The real-time inference system demonstrates low latency, confirming that the model can be deployed in practical web security scenarios without significant performance overhead.

6.3 Limitations of the Study

Despite its effectiveness, the proposed system has certain limitations. The performance of the model is highly dependent on the quality and diversity of the dataset. If the dataset does not include sufficient variations of attack patterns, the model may struggle to generalize to completely new types of attacks.

Another limitation is that the system relies on a predefined set of attack classes. While it can detect known attack categories effectively, it may not fully capture entirely new or evolving attack techniques without retraining. Additionally, although DistilBERT provides a good balance between performance and efficiency, more complex transformer models could potentially achieve higher accuracy at the cost of increased computational requirements.

The system also introduces some computational overhead compared to traditional rule-based firewalls, particularly during model inference. While this overhead is minimized through optimization, it may still be a consideration in extremely high-throughput environments.

6.4 Suggestions for Future Research

Future research can focus on improving the performance and scalability of the AI-WAF system. One potential direction is the use of more advanced transformer architectures or ensemble models to further enhance detection accuracy. Incorporating continual learning techniques can enable the system to adapt to new attack patterns without requiring complete retraining.

Another area of improvement is the expansion of the dataset to include a wider variety of real-world attack scenarios, including zero-day and polymorphic attacks. Integrating threat intelligence feeds and real-time data sources can further enhance the system's ability to detect emerging threats.

The system can also be extended by incorporating hybrid approaches that combine rule-based and AI-based detection mechanisms to improve interpretability and robustness. Additionally, deploying the model using lightweight optimization techniques such as model quantization or distillation can improve inference speed and reduce resource consumption.

Further research can explore integration with cloud-based security platforms and distributed architectures to support large-scale deployments. Enhancing the explainability of model predictions using interpretable AI techniques can also improve trust and usability in real-world applications.

6.5 Summary

In this chapter, the overall contributions, findings, limitations, and future directions of the proposed AI-WAF system have been discussed. The study demonstrates that transformer-based deep learning models, combined with a confidence-aware decision mechanism, provide an effective and intelligent solution for web application security. While certain limitations exist, the proposed system lays a strong foundation for future advancements in AI-driven cybersecurity and highlights the potential of transformer models in protecting modern web applications.

References

[1] N. Thapa, S. S. R. Depuru, and K. Roy, "Transformer-based language models for software vulnerability detection," *Proc. IEEE/ACM Int. Conf. Software Engineering*, 2022, pp. 1–12.

[2] R. Mamede, R. Pinconschi, and F. Abreu, "A transformer-based IDE plugin for vulnerability detection," *Proc. IEEE Int. Conf. Software Maintenance and Evolution*, 2022, pp. 1–10.

[3] E. M. Rudd, M. S. Rahman, and P. Tully, "Transformers for end-to-end InfoSec tasks: A feasibility study," *Proc. IEEE Security and Privacy Workshops*, 2022, pp. 1–10.

[4] J. Darwinkel, "Fingerprinting web servers through transformer-encoded HTTP response headers," *Computers & Security*, vol. 134, pp. 103–118, 2024.

[5] M. Alshomrani, A. Alghamdi, and F. Alsubaei, "Survey of transformer-based malicious software detection systems," *IEEE Access*, vol. 12, pp. 45678–45702, 2024.

[6] S. Latibari, A. Rezaei, and M. K. Rafsanjani, "Transformers: A security perspective," *IEEE Access*, vol. 12, pp. 78901–78925, 2024.

[7] J. De La Torre Parra, A. Pastor, and P. García-Teodoro, "Interpretable federated transformer log learning for cloud threat forensics," *IEEE Trans. Information Forensics and Security*, vol. 17, pp. 3210–3223, 2022.

[8] K. Ravi, S. Menon, and A. Krishnan, "CASB security analytics for encrypted SaaS traffic: A hybrid transformer-based classification framework in enterprise cloud ecosystems," *IEEE Access*, vol. 13, pp. 11234–11249, 2025.

[9] M. Rahman, T. Ahmed, and L. Johnson, "AI-enhanced web application firewalls for protecting United States critical infrastructure against zero-day exploits," *IEEE Access*, vol. 14, pp. 55678–55705, 2026.

[10] R. Black, D. Smith, and J. Carter, "Descriptor: Firewall attack detections and extractions (FADE)," *Proc. USENIX Security Symposium*, 2025, pp. 1–16.

[11] A. Szabó and V. Bilicki, "A new approach to web application security: Utilizing GPT language models for source code inspection," *IEEE Access*, vol. 11, pp. 98765–98780, 2023.

[12] M. Hasan and M. Islam, "High-performance computing architectures for training large-scale transformer models in cyber-resilient applications," *IEEE Access*, vol. 10, pp. 65432–65455, 2022.

[13] A. Ali, "Next-generation AI for advanced threat detection and security enhancement in DNS over HTTPS," *IEEE Access*, vol. 13, pp. 33445–33463, 2025.

[14] M. Ferla, "Enhancing cloud-based web application firewalls with machine learning models for bot detection and HTTP traffic classification," M.S. thesis, Dept. Comput. Sci., Univ. of Padova, Padova, Italy, 2024.

[15] A. Vaswani *et al.*, "Attention is all you need," *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT: A distilled version of BERT—smaller, faster, cheaper and lighter," *Proc. NeurIPS Workshop*, 2019.

[18] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[19] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malicious traffic detection using deep learning methods," *IEEE Access*, vol. 6, pp. 33994–34002, 2018.

[20] Y. Kim, "Convolutional neural networks for sentence classification," *Proc. EMNLP*, 2014, pp. 1746–1751.

[21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[22] Z. Zhang, J. Li, C. Manikopoulos, J. Jorgenson, and J. Ucles, "HIDE: A hierarchical network intrusion detection system using statistical preprocessing and neural network classification," *Proc. IEEE Workshop Information Assurance*, 2001, pp. 85–90.

[23] K. Kendall, "A database of computer attacks for the evaluation of intrusion detection systems," M.S. thesis, MIT, Cambridge, MA, USA, 1999.

[24] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," *Proc. Military Communications and Information Systems Conf.*, 2015, pp. 1–6.

[25] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

APPENDIX

ENCLOURES

Details of Mapping the Project with the Sustainable Development Goals (SDGs)

Fig A.1 Sustainable development goals

SDG 9: Industry, Innovation, and Infrastructure

The proposed AI-WAF system contributes to SDG 9 by enhancing **digital infrastructure security** using advanced Artificial Intelligence techniques. The integration of transformer-based models such as DistilBERT improves the resilience of web applications against cyber attacks. By enabling intelligent threat detection and real-time request analysis, the system promotes innovation in cybersecurity technologies and supports the development of secure and reliable digital systems.

SDG 16: Peace, Justice, and Strong Institutions

This project strongly aligns with SDG 16 by improving **cybersecurity and data protection**. Web applications are widely used in government and institutional systems, and protecting them from cyber threats ensures data integrity, privacy, and trust. The AI-WAF helps prevent unauthorized access, data breaches, and malicious activities, thereby supporting secure digital governance and strengthening institutional systems.

SDG 8: Decent Work and Economic Growth

Cyber attacks can cause significant financial losses and disrupt business operations. The AI-WAF system helps organizations maintain secure web services, thereby ensuring **business continuity and economic stability**. By protecting e-commerce platforms, financial systems, and enterprise applications, the project indirectly contributes to economic growth and secure digital employment environments.

SDG 17: Partnerships for the Goals

The development of this project involves the integration of multiple technologies such as **machine learning, cybersecurity frameworks, web technologies, and open-source tools**. This reflects SDG 17 by promoting collaboration between different technological domains and encouraging knowledge sharing in cybersecurity research and development.

SCREENSHOTS

1. AI-WAF PlayGround

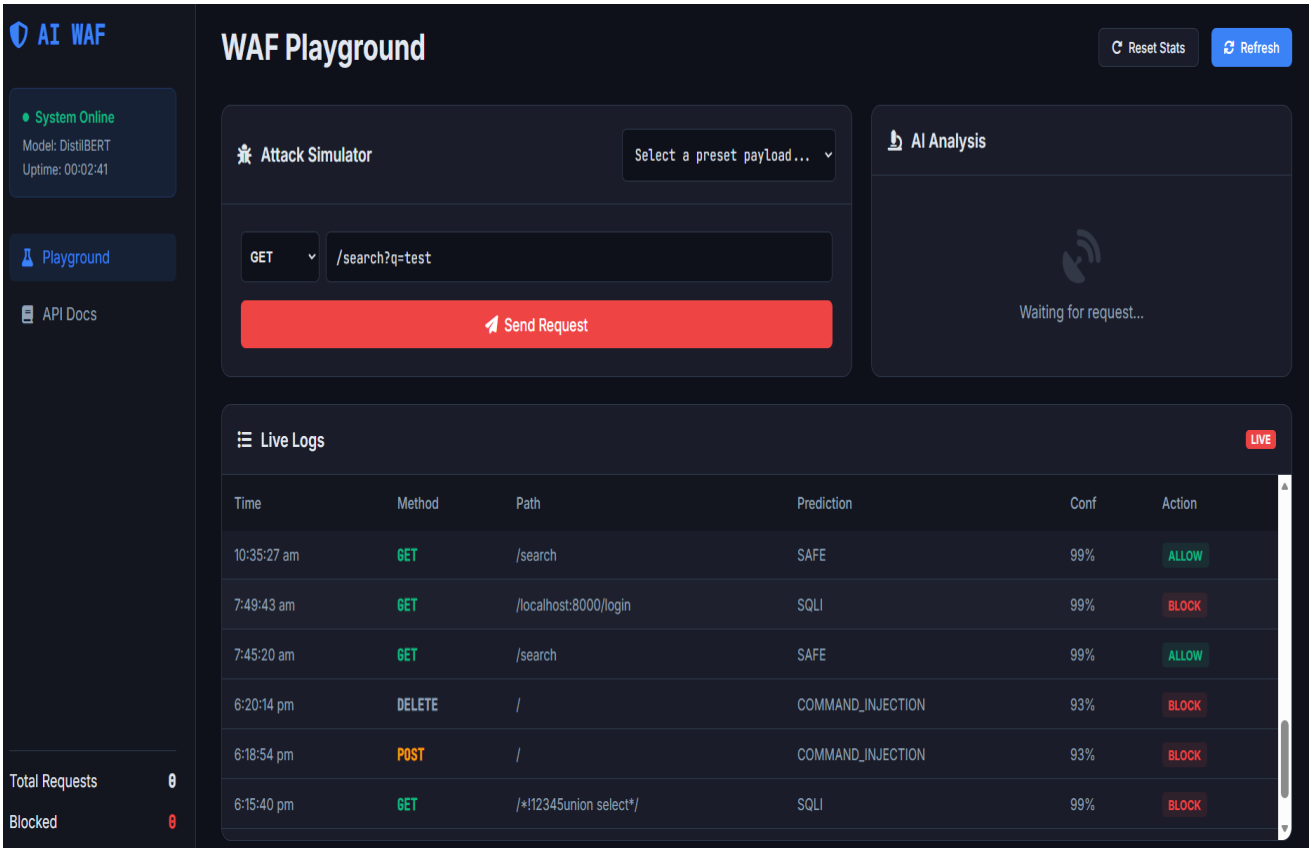
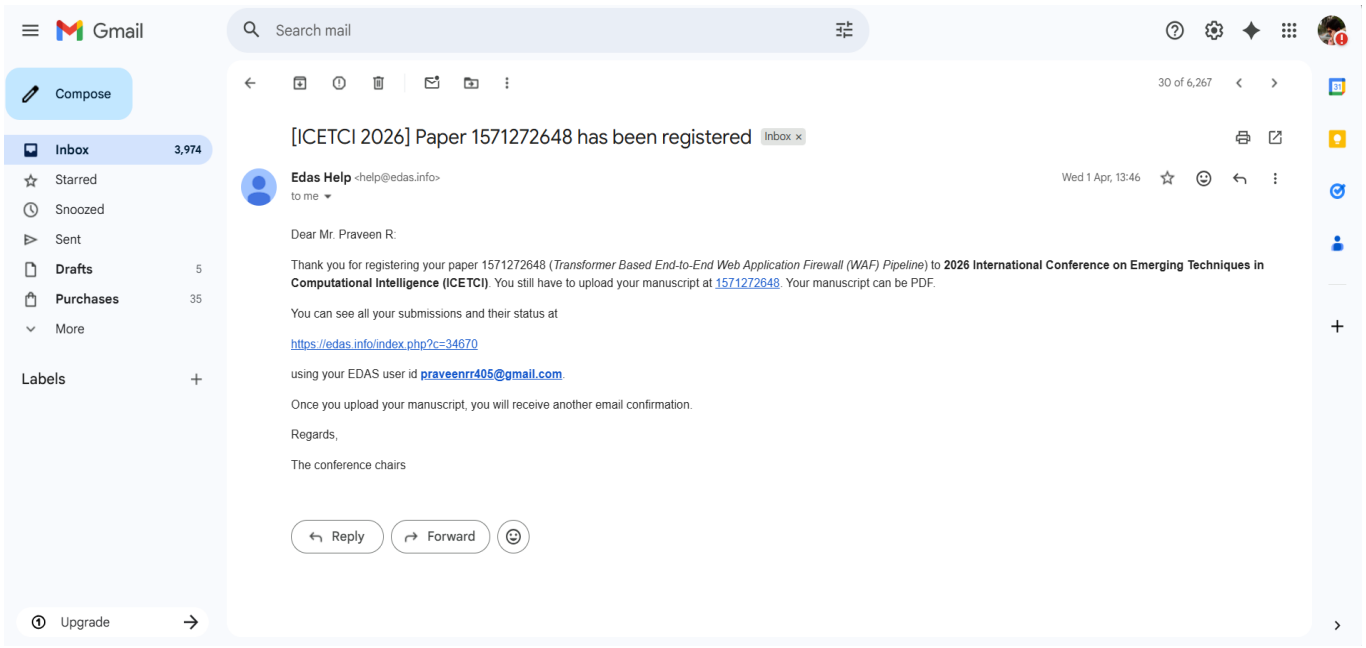
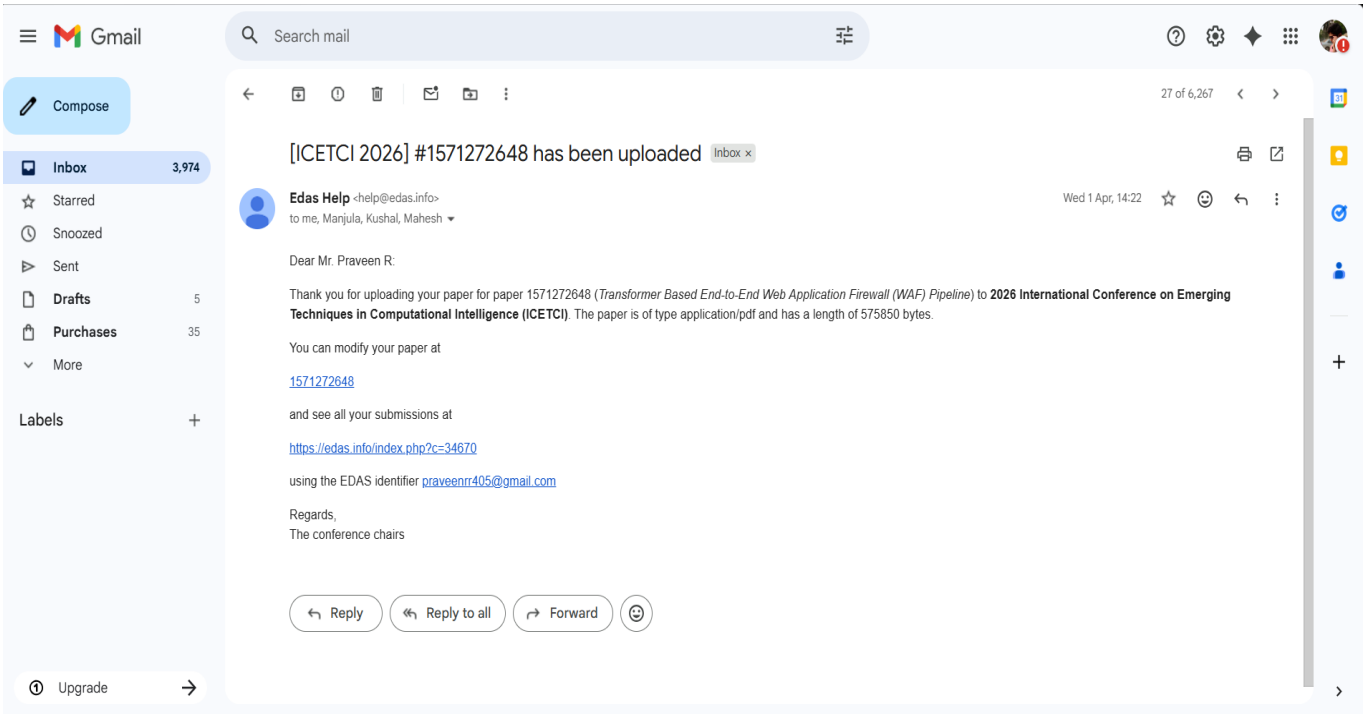


Fig A.2 AI-WAF PLAYGROUND


Gmail
Search mail
Compose
Inbox 3,974
Starred
Snoozed
Sent
Drafts 5
Purchases 35
More
Labels
Upgrade
30 of 6,267
[ICETCI 2026] Paper 1571272648 has been registered
Inbox ×
Edas Help <help@edas.info>
to me
Wed 1 Apr, 13:46
Dear Mr. Praveen R:
Thank you for registering your paper 1571272648 (Transformer Based End-to-End Web Application Firewall (WAF) Pipeline) to 2026 International Conference on Emerging Techniques in Computational Intelligence (ICETCI). You still have to upload your manuscript at 1571272648. Your manuscript can be PDF.
You can see all your submissions and their status at
https://edas.info/index.php?c=34670
using your EDAS user id praveenrr405@gmail.com.
Once you upload your manuscript, you will receive another email confirmation.
Regards,
The conference chairs
Reply
Forward


Gmail
Search mail
Compose
Inbox 3,974
Starred
Snoozed
Sent
Drafts 5
Purchases 35
More
Labels
Upgrade
27 of 6,267
[ICETCI 2026] #1571272648 has been uploaded
Inbox ×
Edas Help <help@edas.info>
to me, Manjula, Kushal, Mahesh
Wed 1 Apr, 14:22
Dear Mr. Praveen R:
Thank you for uploading your paper for paper 1571272648 (Transformer Based End-to-End Web Application Firewall (WAF) Pipeline) to 2026 International Conference on Emerging Techniques in Computational Intelligence (ICETCI). The paper is of type application/pdf and has a length of 575850 bytes.
You can modify your paper at
1571272648
and see all your submissions at
https://edas.info/index.php?c=34670
using the EDAS identifier praveenrr405@gmail.com
Regards,
The conference chairs
Reply
Reply to all
Forward

My pending, active, and accepted papers and proposals

Only papers for upcoming and recently-concluded conferences and journal issues are shown.

Conference	Paper title (details)	Status	Edit	Add and delete authors	Withdraw or unwithdraw	Review manuscript
ICETCI 2026	*Transformer Based End-to-End Web Application Firewall (WAF) Pipeline*	Active (has manuscript)				until Apr 20

el 0

My profile

Name	Mr. Praveen R
EDAS identifier	**2513557**
Type (gender)	student (M)
Affiliation	Computer Science And Technology Presidency University India
Email	praveenrr405@gmail.com
Conflicts of interest (manually added only)	0 not updated

You can subscribe to conferences and journals accepting submissions: